

SIGET - Sistema de Gestión de Torneos

Modelos y Documentación del Software

Docente: Ing. Hely Suarez Marín

Integrantes:

Nelly Fabiola Cano Oviedo

Néstor Iván Granados Valenzuela

Fecha: Octubre / 2025

Problema

Los torneos deportivos académicos y aficionados suelen gestionar su información con hojas de cálculo, formularios dispersos o mensajería instantánea. A medida que crece el número de equipos, jugadores y partidos, también aumenta la **complejidad operativa**: programar encuentros sin choques de horario, asignar árbitros disponibles, registrar resultados de forma confiable y consultar estadísticas básicas.

Este manejo **manual y descentralizado** provoca errores de transcripción (marcadores mal digitados, equipos repetidos), **inconsistencia de datos** (jugadores duplicados, árbitros inhabilitados asignados por error) y **poca trazabilidad** (no queda claro quién cambió un resultado ni cuándo). La ausencia de un **modelo único** limita la generación de estadísticas confiables (partidos jugados, victorias, empates, derrotas) y dificulta la rendición de cuentas. En consecuencia, se afectan la **transparencia** del torneo y la **experiencia** de participantes y organizadores.

Solución

Se propone una **plataforma web simple y centralizada** de Gestión de Torneos Deportivos que estandarice:

1. **Registro de equipos y jugadores,**
2. **Programación de partidos** (fecha/hora/ venue),
3. **Asignación de árbitros** disponibles,
4. **Registro de resultados,** y
5. **Estadísticas** básicas por equipo.

El acceso se realiza con roles mínimos (Administrador, Árbitro, Espectador) para **delimitar responsabilidades** y reducir errores. El sistema define **reglas de negocio** que evitan inconsistencias (por ejemplo, no se puede programar un partido con el mismo equipo como

local y visitante, o cerrar un partido played con goles negativos). Toda la información queda persistida en una **BD relacional** con **integridad referencial**.

Impacto

1. **Eficiencia y trazabilidad:** Disminuye tiempos de registro y elimina duplicidades. Cada cambio queda asociado a un usuario/fecha, reforzando el control.
2. **Transparencia del torneo:** Estadísticas generadas a partir de resultados **verificables** en la BD; se reducen conflictos por errores humanos.
3. **Escalabilidad académica:** La documentación (UML + BD normalizada) deja una base sólida para futuras extensiones (múltiples torneos, fases, sanciones, tablas de posiciones).

1. Resumen del Problema

El manejo manual y distribuido de información en torneos deportivos provoca **inconsistencias**, **pérdida de trazabilidad** y **dificulta estadísticas confiables**. Esto impacta la calidad organizativa y la experiencia de equipos y árbitros.

2. Propuesta de solución (plataforma mínima de torneo)

Se propone un **MVP** que cubra el ciclo esencial: inscripción de equipos/jugadores → programación y arbitraje → registro de resultados → estadísticas. La solución se documenta completamente (UML + BD) para una implementación posterior.

Alcance (MVP)

- Registrar **Equipos** (nombre, ciudad/entrenador) y **Jugadores** (básicos y pertenencia a un equipo).
- Registrar **Árbitros** (licencia, contacto, estado activo).
- **Programar partidos** con fecha y hora, asegurando que **local ≠ visitante**.
- **Asignar árbitro** disponible.
- **Registrar resultados** (goles local/visitante) y marcar estado del partido como played.
- Generar **estadísticas básicas**: jugados, ganados, empatados y perdidos por equipo.

Roles del sistema y sus responsabilidades

Actor	Funciones principales
-------	-----------------------

Administrador	Gestiona equipos, jugadores y árbitros; programa partidos; registra resultados; consultas estadísticas.
Árbitro	(Opcional) Confirma validez del resultado o reporta incidentes.
Espectador	Consultas estadísticas y resultados publicados.

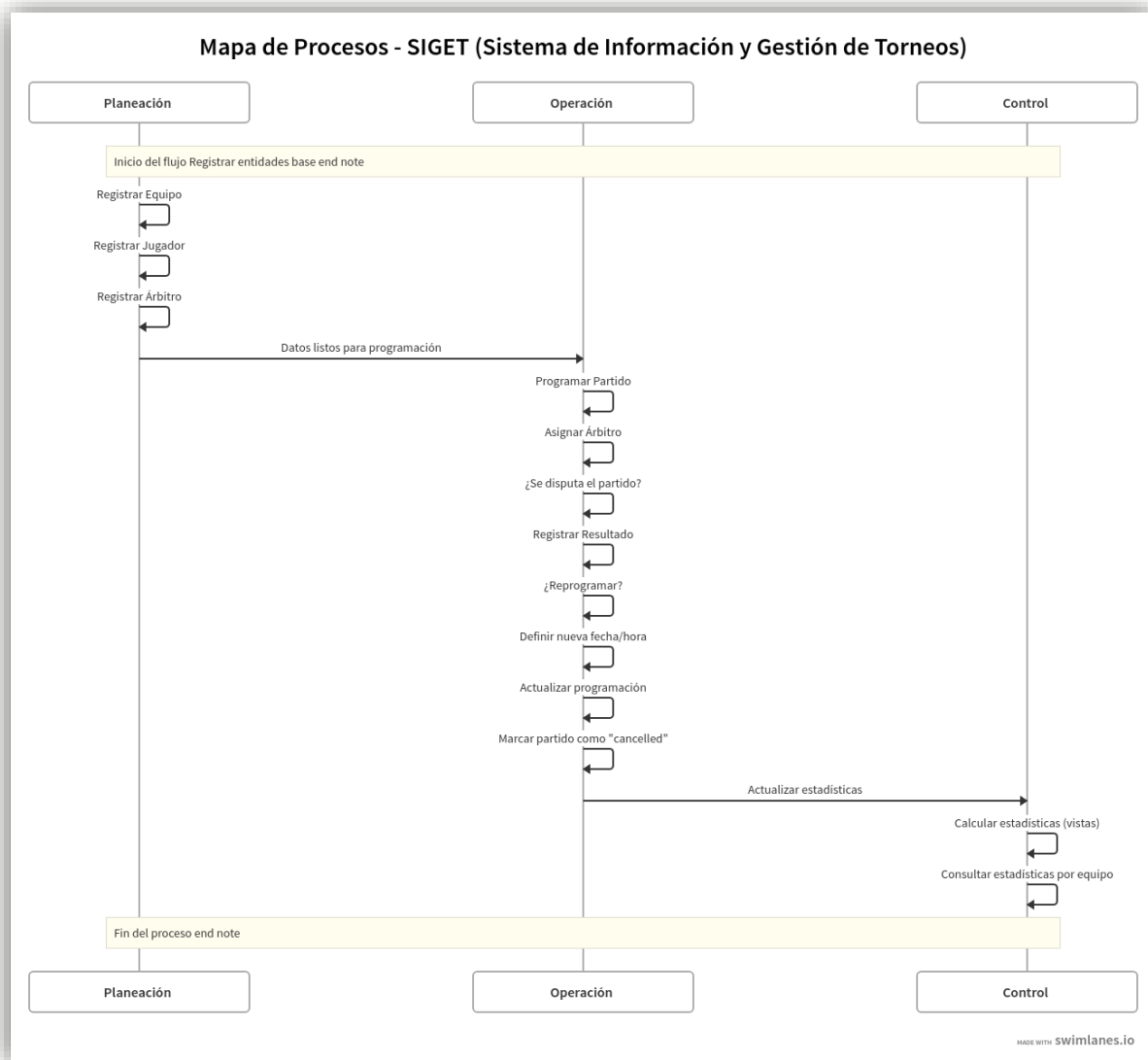
Reglas clave del sistema

- Un partido **no puede** tener el mismo equipo como local y visitante.
- Un partido **played** debe registrar **goles ≥ 0** .
- Un árbitro **inactivo** no puede asignarse.
- Las **estadísticas** se **derivan** de los partidos; no se almacenan acumulados (evita inconsistencias).

3. Descripción general del proceso

Primero se registran equipos, jugadores y árbitros. Luego se programan los partidos y se asigna un árbitro. Una vez disputado el encuentro, se registra el resultado, que luego alimenta las estadísticas. Este flujo corresponde al mapa de procesos general del sistema (Planeación → Operación →

Control).



4. Validación del modelo

Los flujos propuestos fueron contrastados con prácticas reales: las planillas manuales suelen causar errores, pérdida de control y ausencia de trazabilidad. El sistema SIGET corrige estos problemas mediante estructura relacional, reglas de negocio, restricciones, validaciones y vistas para reportes.

5. Diagramas UML realizados

5.1 Casos de Uso

□ **Propósito:** Mostrar las funcionalidades que ofrece SIGET desde la perspectiva de cada actor, sin entrar en detalles técnicos.

□ **Qué representa:**

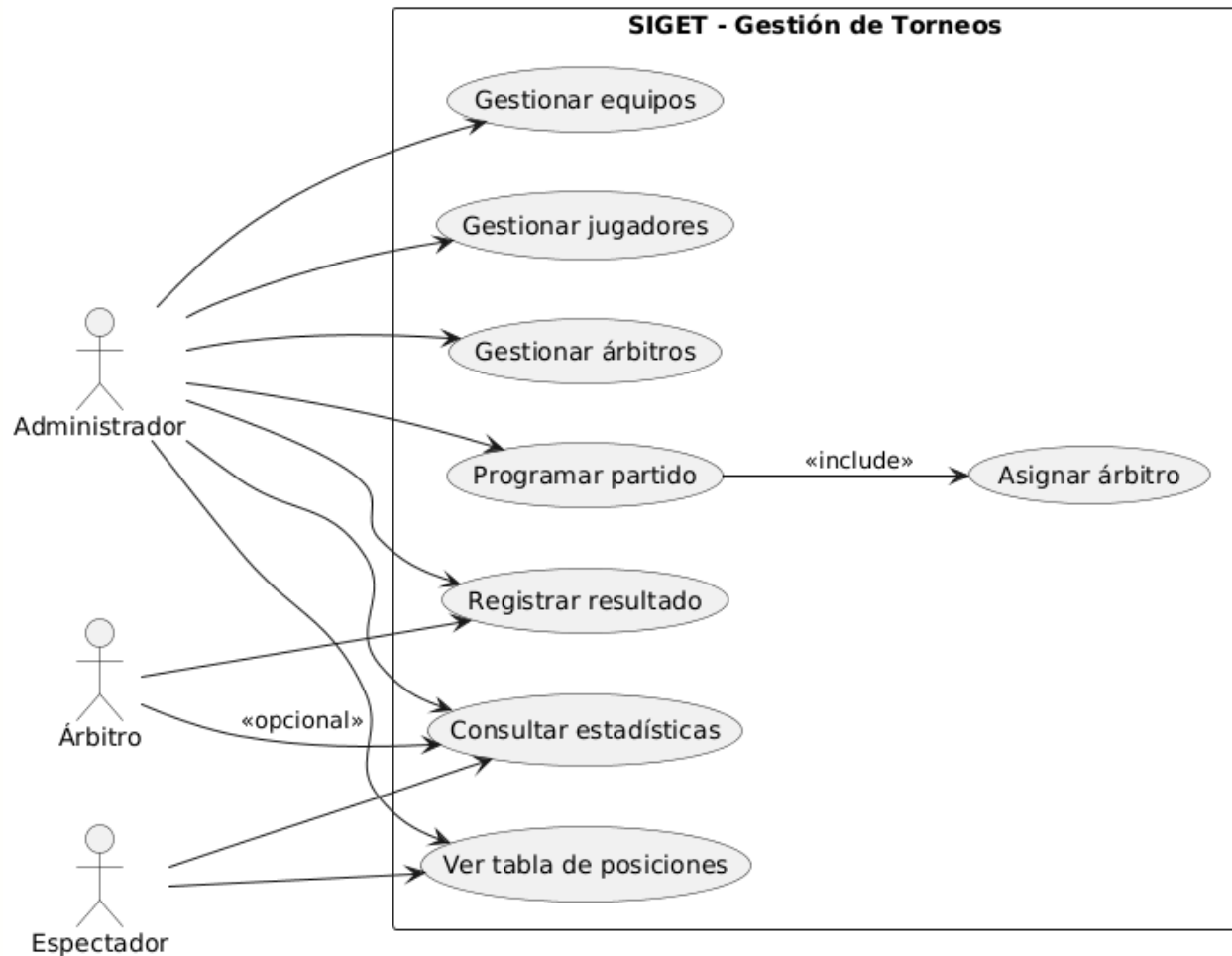
- Administrador: gestiona equipos, jugadores, árbitros, partidos y resultados.
 - Árbitro: registra resultados y consulta estadísticas (opcional).
 - Espectador: consulta estadísticas y tabla de posiciones.
- Casos clave: Gestionar equipos/jugadores/árbitros, Programar partido, Asignar árbitro, Registrar resultado, Consultar estadísticas y Ver tabla de posiciones.

□ **Cómo leerlo:** Cada óvalo es un caso de uso; las líneas indican qué actor lo ejecuta. El estereotipo «include» indica dependencia obligatoria (Programar partido → Asignar árbitro).

□ **Razón de diseño:** Representa el alcance central del MVP evitando funciones avanzadas para mantener claridad académica.

□ **Relación con BD:** Cada caso se vincula a entidades y restricciones de la base (equipos, árbitros, matches, validaciones de estado y goles).

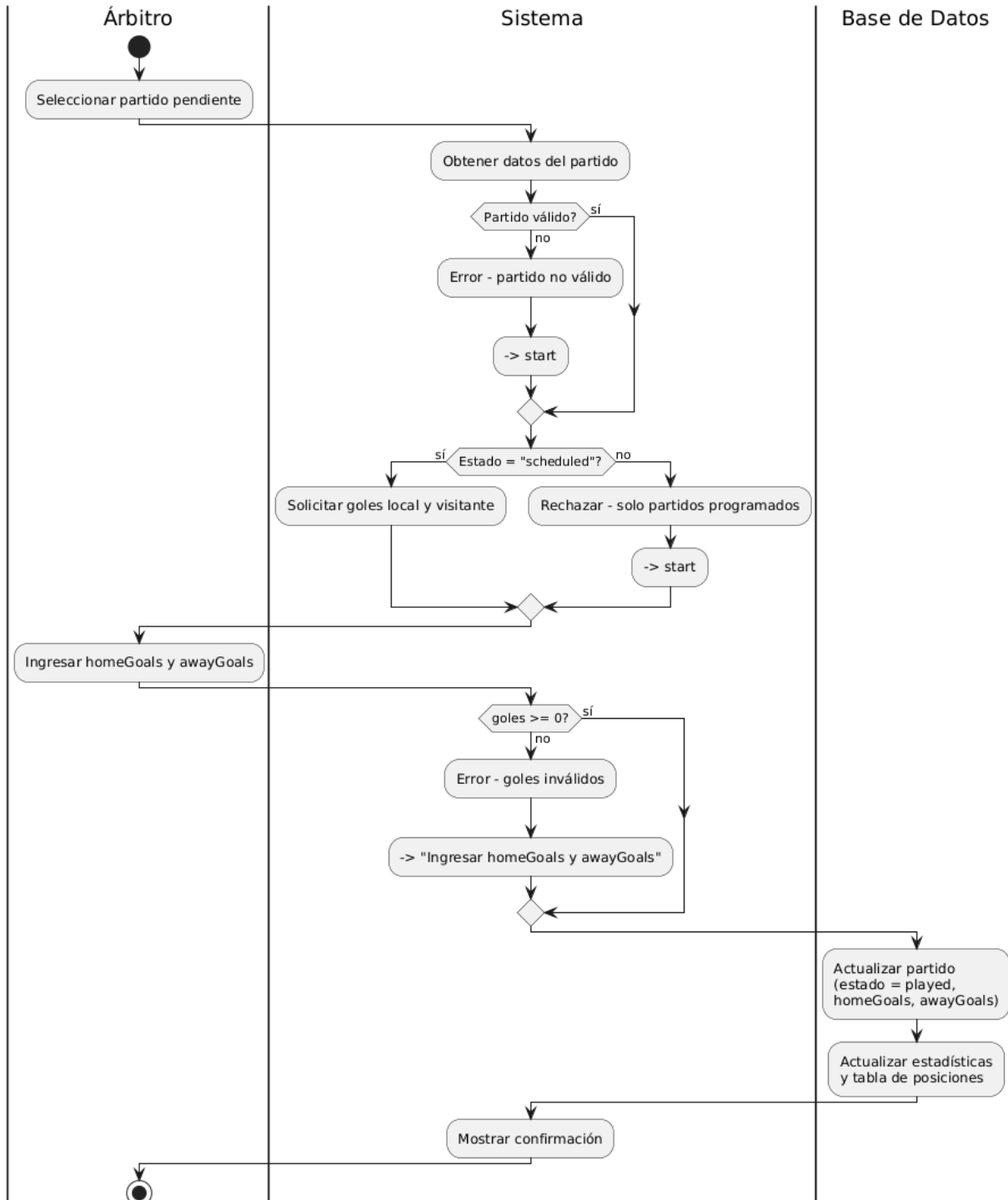




5.2 Actividades (Registrar Resultado)

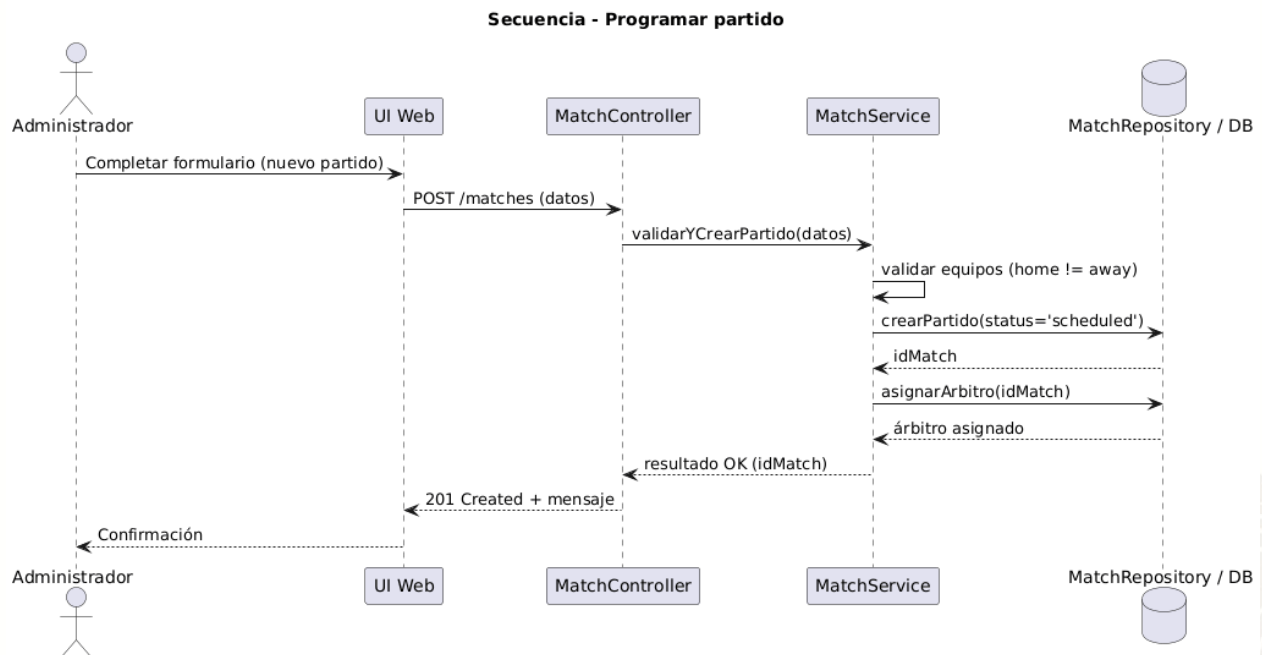
- **Propósito:** Mostrar el flujo paso a paso con validaciones explícitas.
- **Qué representa:** El árbitro selecciona un partido, se valida que esté scheduled, se ingresan y verifican los goles (≥ 0), se actualiza a played y se recalculan estadísticas. Si el partido no existe o no está en scheduled, se rechaza.
- **Cómo leerlo:** Actividades en rectángulos, decisiones en rombos, flechas que marcan el flujo y bucles para errores.
- **Razón de diseño:** Garantizar consistencia antes de cerrar un partido.
- **Relación con BD:** Cambia matches.status, home_goals y away_goals, lo que complementa la actualización de estadísticas.

Actividad - Registrar resultado



5.3 Secuencia (Programar Partido)

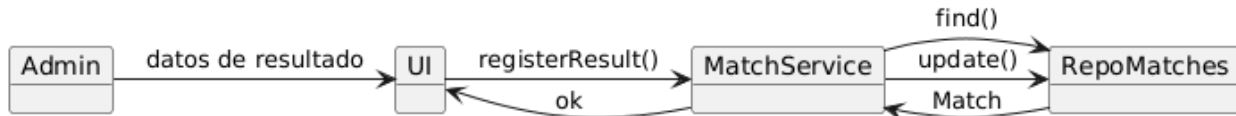
- **Propósito:** Mostrar cómo colaboran UI, controlador, servicio y repositorio.
- **Qué representa:** Validación de equipos, creación del partido en scheduled y asignación del árbitro.
- **Cómo leerlo:** Los mensajes muestran validación → inserción → confirmación.
- **Razón de diseño:** Evidenciar el flujo completo sin complejidad adicional.
- **Relación con BD:** Inserción en matches y asignación en referee_id.



5.4 Comunicación (Registrar Resultado)

- **Propósito:** Mostrar cómo se comunican componentes al registrar un resultado.
- **Qué representa:** UI envía datos → MatchService valida → RepoMatches actualiza partido → UI recibe confirmación.
- **Cómo leerlo:** Flujo izquierda–derecha mostrando solicitudes y respuestas.
- **Razón de diseño:** Enfatiza que la validación es responsabilidad del servicio, no de la UI.
- **Relación con BD:** Actualización en matches usando update().

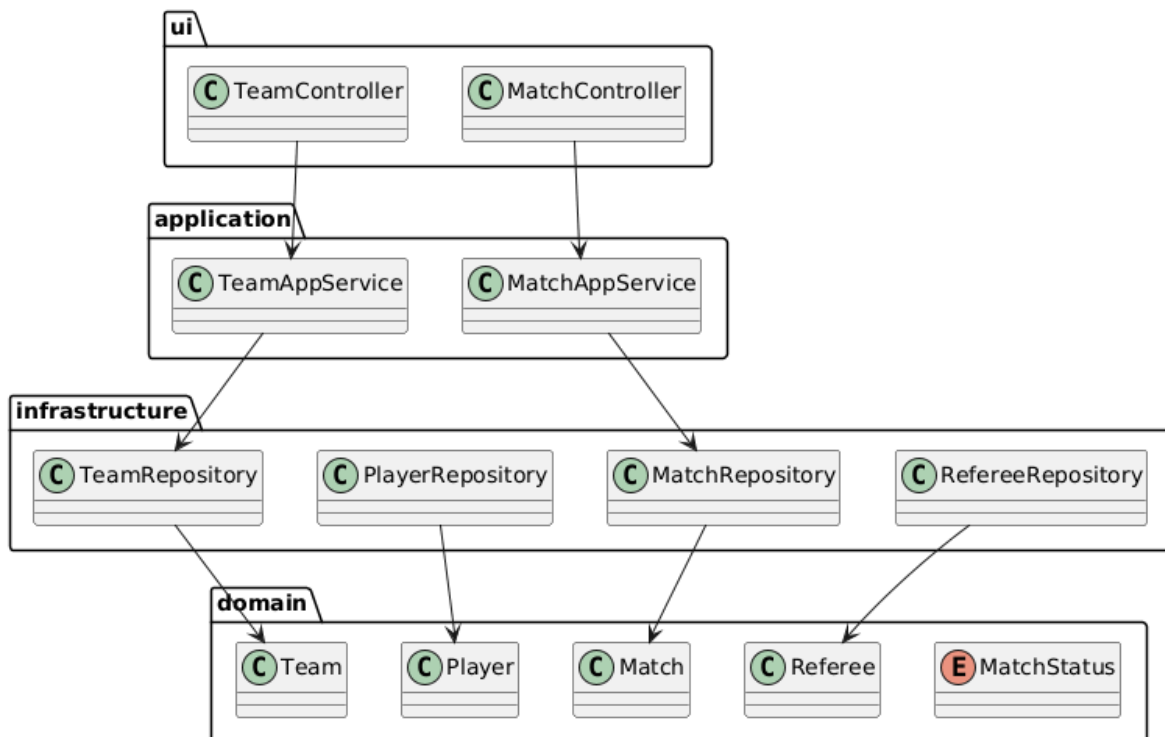
Comunicación: Registrar Resultado



5.5 Paquetes

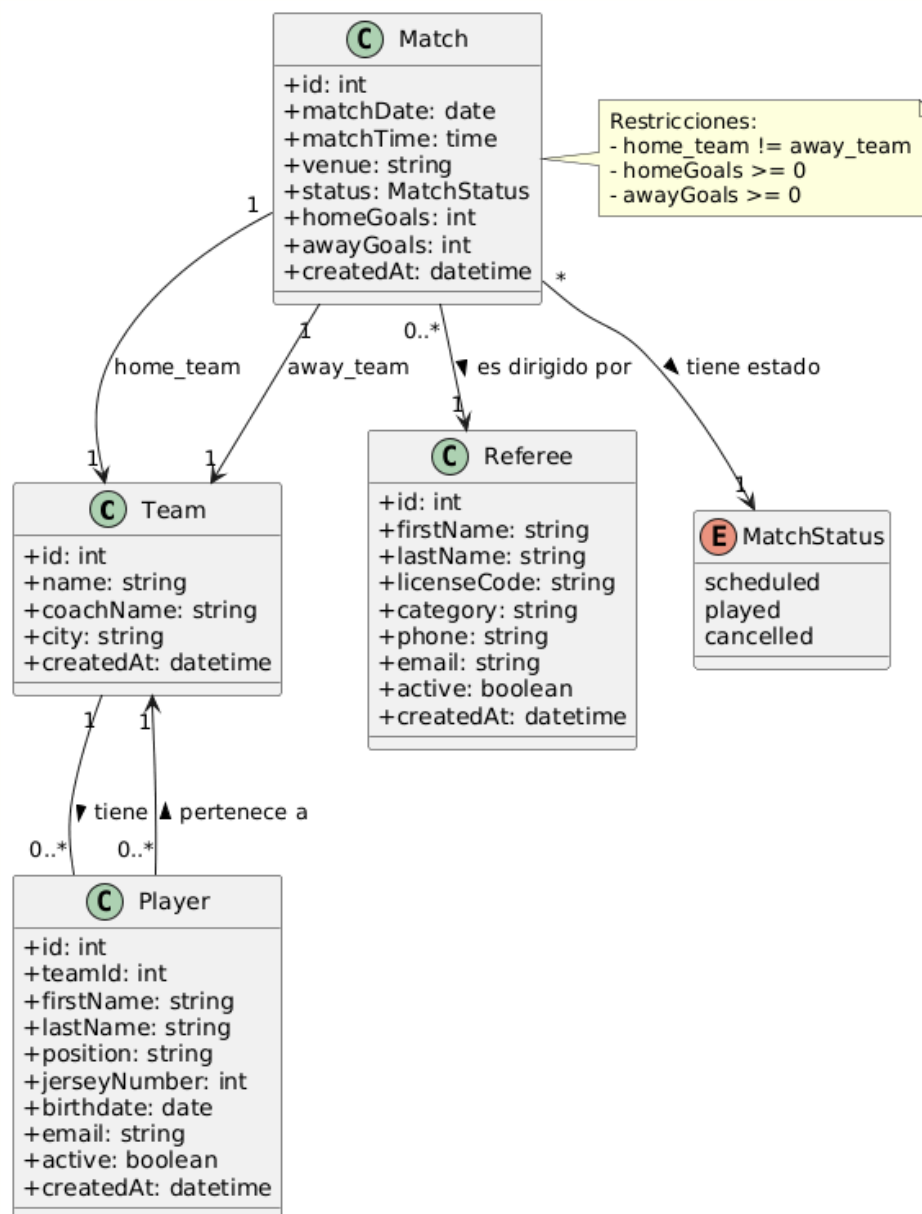
- **Propósito:** Organizar SIGET en capas lógicas para separar responsabilidades.
- **Qué representa:**
 - ui: controladores y entrada de usuario
 - application: casos de uso
 - domain: entidades del negocio
 - infrastructure: repositorios y acceso a BD
- **Cómo leerlo:** Las flechas indican dependencias entre capas.
- **Razón de diseño:** Mantener modularidad y claridad entre lógica, datos e interfaz.
- **Relación con BD:** La capa infrastructure es la encargada del acceso.

Paquetes - Arquitectura lógica SIGET



5.6 Clases

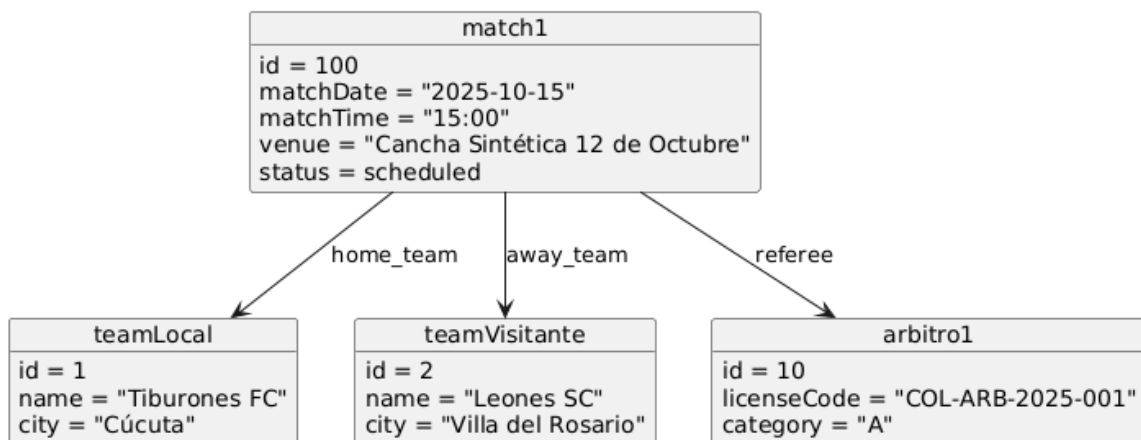
- **Propósito:** Representar la estructura estática del dominio.
- **Qué representa:** Team, Player, Match, Referee y MatchStatus (enum).
- **Cómo leerlo:** Rectángulos = clases, líneas = asociaciones, multiplicidades = cardinalidad.
- **Razón de diseño:** Modelo mínimo, preciso y coherente con el relacional.
- **Relación con BD:** Traducción directa a teams, players, referees, matches, incluyendo PK, FK y restricciones.



5.7 Objetos

- **Propósito:** Mostrar un ejemplo real del modelo.
- **Qué representa:** Un partido concreto con equipos y árbitro asociados.
- **Cómo leerlo:** Atributos con valores reales; flechas representan vínculos.
- **Razón de diseño:** Aterriza el modelo a datos reales.
- **Relación con BD:** Corresponde a registros específicos en las tablas.

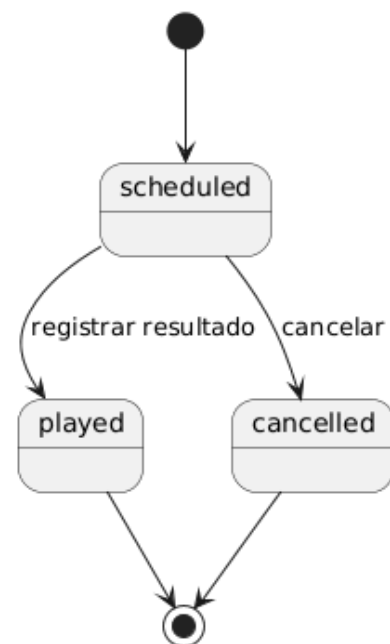
Diagrama de Objetos (Ejemplo)



5.8 Estados (Partido)

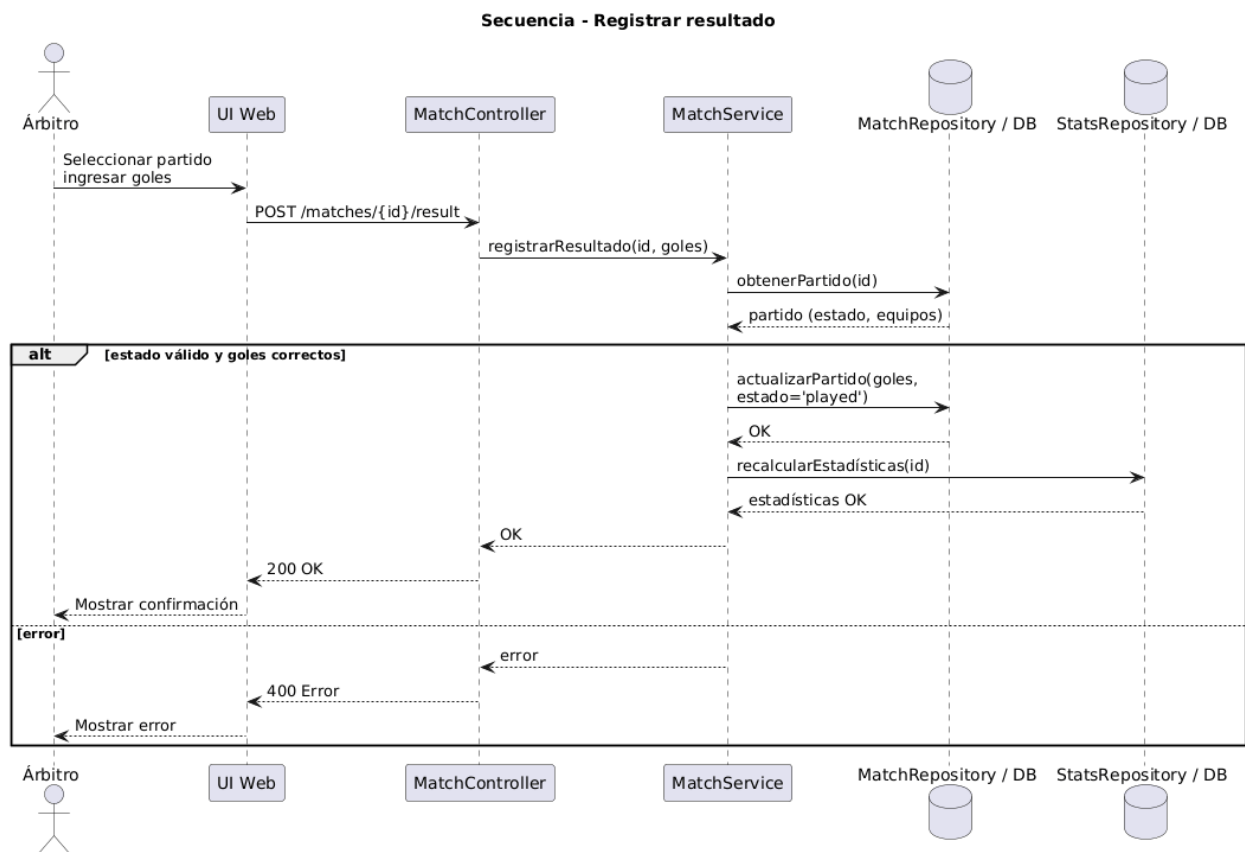
- **Propósito:** Mostrar su ciclo de vida.
- **Qué representa:** scheduled → played o scheduled → cancelled.
- **Cómo leerlo:** Estado inicial, transiciones y fin.
- **Razón de diseño:** Garantizar reglas de flujo válidas.
- **Relación con BD:** Controla el valor de matches.status.

Estados: Partido



5.9 Secuencia (Registrar Resultado)

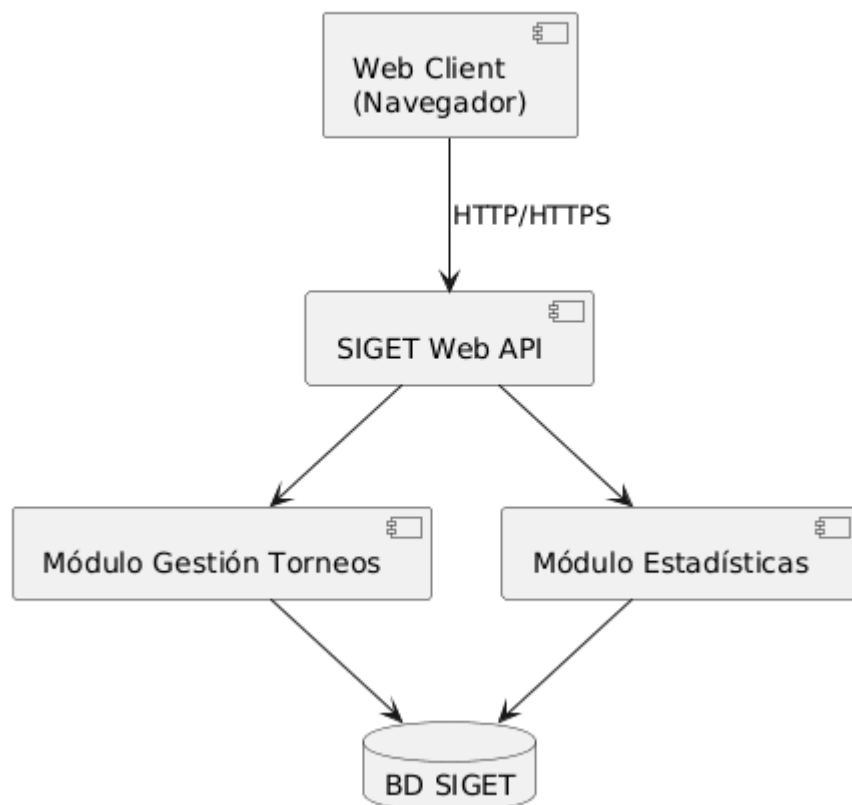
- **Propósito:** Modelar la interacción completa para registrar un resultado.
- **Qué representa:** Ingreso de goles → validación → actualización → estadísticas.
- **Cómo leerlo:** Mensajes secuenciales que muestran cada paso.
- **Razón de diseño:** Garantizar exactitud del proceso.
- **Relación con BD:** Cambios en matches y cálculo de estadísticas.



5.10 Componentes

- **Propósito:** Mostrar cómo se divide SIGET en componentes.
- **Qué representa:** Web Client → API → módulos → BD.
- **Cómo leerlo:** Flechas = canales de comunicación.
- **Razón de diseño:** Desacoplamiento.
- **Relación con BD:** Módulos consultan/escriben en la BD.

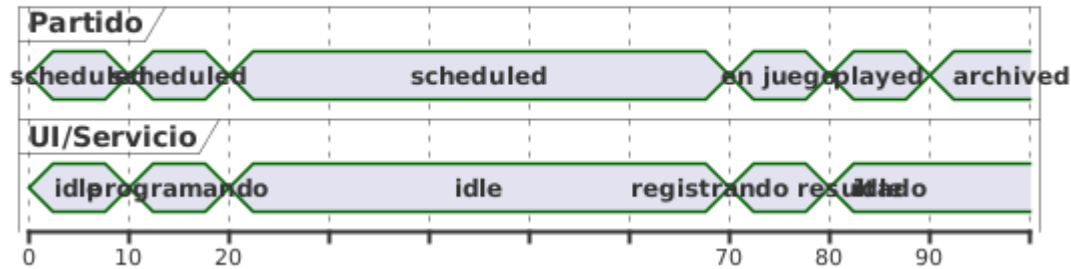
Componentes - SIGET



5.11 Tiempo (Timing)

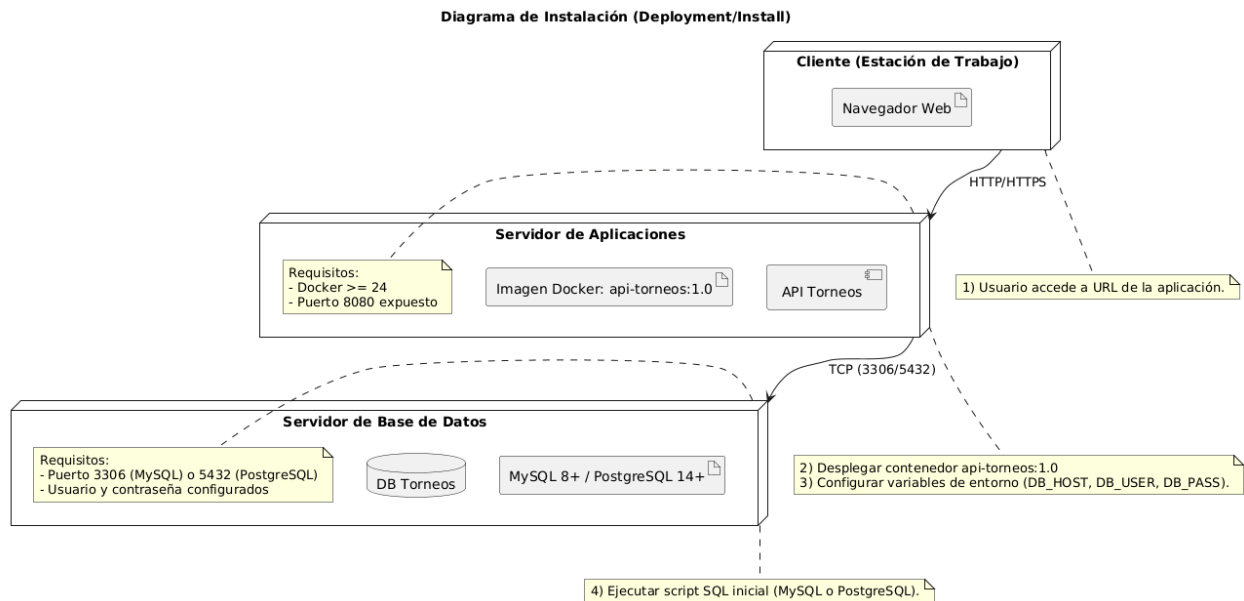
- **Propósito:** Mostrar cambios de estado en el tiempo.
- **Qué representa:** Estados del partido y comportamiento de la UI/Servicio.
- **Cómo leerlo:** Cada tramo representa un estado temporal.
- **Razón de diseño:** Visualizar etapas del partido.
- **Relación con BD:** Cada estado corresponde a un valor en matches.status.

Diagrama de Tiempo (Timing): Ciclo de un Partido



5.12 Instalación (Deployment/Install)

- **Propósito:** Describir el despliegue del sistema.
- **Qué representa:** Cliente por navegador, servidor de aplicaciones con Docker, servidor de BD.
- **Cómo leerlo:** Cada nodo es un servidor; flechas = comunicación.
- **Razón de diseño:** Indicar pasos mínimos para ejecutar SIGET.
- **Relación con BD:** El servidor de aplicaciones se conecta a DB Torneos usando variables de entorno.



6. Diagramas de Base de Datos (no UML – PlantUML ER/IE)

6.1 Modelo Entidad-Relación (ER)

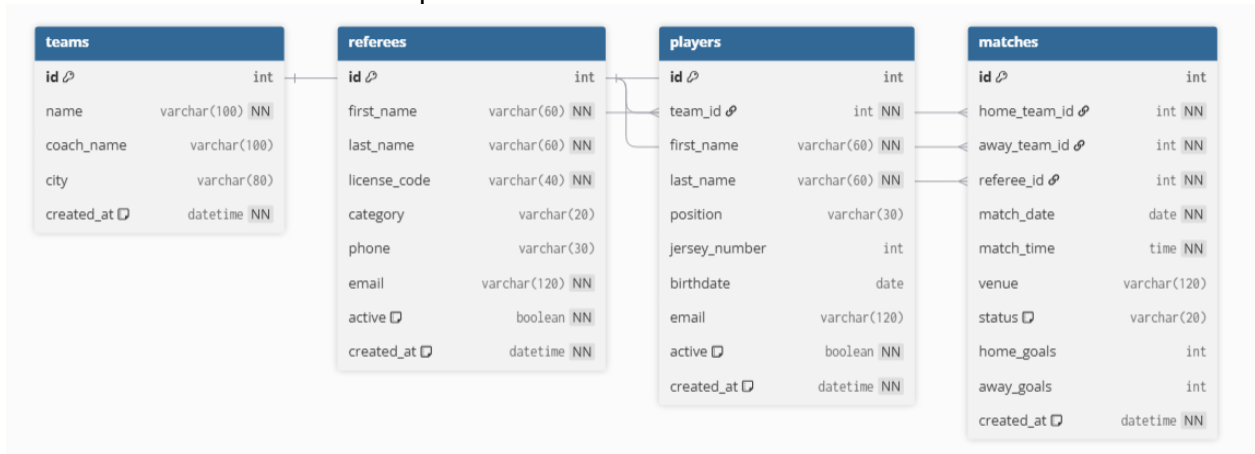
El modelo ER resume las entidades centrales del sistema y sus relaciones esenciales:

- **teams**
- **players**
- **referees**
- **matches**

Relaciones principales:

- **Team 1 — N Player:** un equipo puede tener muchos jugadores.
- **Match N — 1 Referee:** cada partido tiene un árbitro asignado.
- **Match → Team (home/away):** un partido referencia dos equipos distintos (local y visitante).

Estas relaciones definen la base para las llaves foráneas en el modelo relacional.



6.2 Modelo Relacional

Traducción del modelo ER a tablas con claves primarias, foráneas y restricciones:

- **teams**
(id PK, name UNIQUE, coach_name, city, created_at)
- **players**
(id PK, team_id FK→teams, first_name, last_name, position, jersey_number, birthdate, email, active, created_at)
- **referees**
(id PK, first_name, last_name, license_code UNIQUE, category, phone, email UNIQUE, active, created_at)

- **matches**

(id PK, home_team_id FK→teams, away_team_id FK→teams, referee_id FK→referees, match_date, match_time, venue, status, home_goals, away_goals, created_at, CHECK(home_team_id <> away_team_id))

Este modelo asegura integridad referencial y evita inconsistencias comunes en la gestión manual.

7. Normalización (hasta 3FN)

Aplicada sobre todas las tablas principales:

- **1FN:** Atributos atómicos; sin campos multivalor.
- **2FN:** Llaves primarias simples → no hay dependencias parciales.
- **3FN:** No existen dependencias transitivas entre atributos no clave.
Las estadísticas no se almacenan: se **derivan** de la tabla matches para evitar inconsistencias.

8. Vistas SQL

- **vw_team_matches:** une equipos y partidos e identifica si el equipo actúa como local o visitante.
- **vw_team_stats:** calcula jugados, ganados, empatados y perdidos a partir de partidos played.

Nota: Las vistas no tienen FK; su relación con tablas base puede representarse mediante líneas punteadas en diagramas técnicos.

9. Guía breve de ejecución en PostgreSQL (DBeaver)

1. Conéctate a la base **postgres** y ejecuta:

```
DROP DATABASE IF EXISTS tournament_db WITH (FORCE);  
CREATE DATABASE tournament_db;
```

2. Cambia el editor a **tournament_db** y ejecuta el resto del script (**sin \c**): creación de tablas, tipo match_status, índices y vistas.

